CS2023 - Programación III

Ejercicio de Programación Concurrente

Rubén Rivas Medina

Ejercicio 1: Transformación y reducción — Future + Promise · Intermedio

Un sistema puede sumar importes de ventas, calcular métricas de sensores o agregar puntuaciones. En todos los casos aplica una transformación a cada elemento y combina los resultados. Diseñe una función reutilizable que reciba esas operaciones como argumentos.

Algoritmo y estrategia paralela

Implemente una variante paralela de `std::transform_reduce`. Divida el rango en bloques contiguos. Cada trabajador transforma y reduce su bloque, y entrega el subtotal mediante `std::promise<T>`. El coordinador recoge los futuros, combina los subtotales y aplica `init` una sola vez. Para un rango vacío, devuelve `init`.

Interfaz

```
template<class ForwardIt, class T, class UnaryOp, class BinaryOp>
T parallel_transform_reduce(
    ForwardIt first, ForwardIt last, T init,
    UnaryOp transform, BinaryOp combine,
    std::size_t workers);
```

Restricciones

`ForwardIt` es al menos un iterador hacia adelante. `transform(*it)` debe poder convertirse en `T`; `combine(T, T)` devuelve `T`, es asociativa y admite combinar subtotales en orden de bloque. No suponga que `T` tiene constructor por defecto ni identidad conocida: inicie cada subtotal con el primer elemento transformado de su bloque. `workers == 0` lanza `std::invalid_argument`. Cree `min(workers, n)` hilos para un rango no vacío de `n` elementos: con cuatro ventas y `workers == 8`, cree cuatro hilos, uno por venta. Copie las operaciones para cada trabajador y suponga que sus copias pueden ejecutarse a la vez. Propague mediante `set_exception` cualquier error ocurrido en un trabajador. Consuma todos los futuros y espere todos los hilos antes de propagar un error. No use `std::async` ni `std::transform_reduce` para implementar el algoritmo.

Casos de prueba

Importes con distintos números de trabajadores.

```
std::vector<int> ventas = {2, 1, 3, 0};
auto importe = [](int n) { return n * 125; };
for (std::size_t h : {1u, 2u, 8u})
    assert(parallel_transform_reduce(
        ventas.begin(), ventas.end(), 10,
        importe, std::plus<int>{}, h) == 760);
```

Otro tipo de iterador y rango vacío.

```
std::list<std::string> palabras = {"sol", "luna", "mar"};
auto longitud = [](const auto& s) { return s.size(); };
assert(parallel_transform_reduce(
    palabras.begin(), palabras.end(), std::size_t{0},
    longitud, std::plus<std::size_t>{}, 3) == 10);
assert(parallel_transform_reduce(
    palabras.end(), palabras.end(), 7,
    [](const auto&) { return 1; }, std::plus<int>{}, 2) == 7);
```

Excepción durante la transformación.

```
bool fallo = false;
try {
    (void)parallel_transform_reduce(
        ventas.begin(), ventas.end(), 0,
        [](int x) { if (x == 3) throw std::runtime_error("dato");
                    return x; }, std::plus<int>{}, 2);
} catch (const std::runtime_error&) { fallo = true; }
assert(fallo);
```

CS2023 - Programación III

Ejercicio de Programación Concurrente

Rubén Rivas Medina

Ejercicio 2: Copia filtrada estable — Future + Promise · Intermedio

En un flujo de registros puede interesar conservar alertas graves; en un catálogo, productos disponibles. El criterio y el tipo de elemento deben ser elegidos por quien llama, y la salida debe mantener el orden original.

Algoritmo y estrategia paralela

Implemente una variante paralela de `std::copy_if`. Cada trabajador examina un bloque contiguo y entrega un vector local mediante `std::promise<std::vector<Value>>`. El coordinador consume los futuros en orden de bloque y escribe los valores aceptados en `dest`. El orden de finalización de los hilos no debe alterar la salida.

Interfaz

```
template<class ForwardIt, class OutputIt, class Pred>
OutputIt parallel_copy_if(
    ForwardIt first, ForwardIt last,
    OutputIt dest, Pred pred, std::size_t workers);
```

Restricciones

`ForwardIt` es al menos un iterador hacia adelante y su tipo de valor es copiable. `OutputIt` admite escribir los valores seleccionados; el llamador garantiza espacio suficiente o usa un inserter (`std::inserter` o `std::back_inserter`). La función devuelve el iterador de salida posterior al último valor escrito. El predicado debe poder copiarse para cada trabajador y sus copias deben poder ejecutarse a la vez. `workers == 0` lanza `std::invalid_argument`; el rango vacío no escribe nada. No cree más trabajadores que elementos. Si falla un predicado, entregue la excepción con la promesa, espere todos los hilos y propáguela antes de escribir en `dest`. No use `std::copy_if` en la implementación.

Casos de prueba

Filtrado estable de enteros.

```
std::vector<int> numeros = {7, 2, 9, 4, 6, 1};
for (std::size_t h : {1u, 3u, 10u}) {
    std::vector<int> pares;
    parallel_copy_if(numeros.begin(), numeros.end(),
                     std::back_inserter(pares),
                     [](int x) { return x % 2 == 0; }, h);
    assert((pares == std::vector<int>{2, 4, 6}));
}
```

Predicado sobre una lista de cadenas.

```
std::list<std::string> etiquetas = {"ok", "error", "aviso", "error"};
std::vector<std::string> errores;
parallel_copy_if(etiquetas.begin(), etiquetas.end(),
                 std::back_inserter(errores),
                 [](const auto& s) { return s == "error"; }, 2);
assert((errores == std::vector<std::string>{"error", "error"}));
```

Rango vacío e iterador de salida.

```
std::vector<int> salida = {99};
auto fin = parallel_copy_if(numeros.end(), numeros.end(),
    salida.begin(), [](int) { return true; }, 4);
assert(fin == salida.begin() && salida == std::vector<int>{99});
```

CS2023 - Programación III

Ejercicio de Programación Concurrente

Rubén Rivas Medina

Ejercicio 3: Prefijos acumulados — Async · Intermedio

Los acumulados sirven para ventas diarias, distancias recorridas y concatenación de eventos. El tipo de dato y la operación de combinación deben elegirse en cada llamada.

Algoritmo y estrategia paralela

Implemente una variante paralela de `std::inclusive_scan`. Lance tareas con `std::async(std::launch::async, ...)` para calcular los prefijos locales y el total de cada bloque. Calcule después el acumulado anterior a cada bloque y lance una segunda fase asíncrona para ajustar sus prefijos. Escriba la salida final en orden, por medio de `dest`.

Interfaz

```
template<class ForwardIt, class OutputIt, class BinaryOp>
OutputIt parallel_inclusive_scan(
    ForwardIt first, ForwardIt last,
    OutputIt dest, BinaryOp op, std::size_t tasks);
```

Restricciones

`ForwardIt` es al menos un iterador hacia adelante y su tipo de valor es copiable. `op(T, T)` devuelve `T` y es asociativa; no necesita ser conmutativa. Se conserva el orden de operandos de izquierda a derecha. `OutputIt` dispone de espacio suficiente o es un inserter. Devuelva el iterador posterior al último valor escrito. `tasks == 0` lanza `std::invalid_argument`; un rango vacío no escribe nada. Cree a lo sumo `min(tasks, n)` tareas por fase. Copie la operación para cada tarea y suponga que esas copias pueden ejecutarse a la vez. Consuma todos los futuros antes de escribir en `dest` o propagar un error. No use `std::inclusive_scan` ni `std::partial_sum` en la implementación.

Casos de prueba

Acumulado numérico con varios repartos.

```
std::vector<int> dias = {100, 0, 75, 25, 300};
for (std::size_t t : {1u, 2u, 8u}) {
    std::vector<int> acumulado;
    parallel_inclusive_scan(dias.begin(), dias.end(),
        std::back_inserter(acumulado), std::plus<int>{}, t);
    assert((acumulado == std::vector<int>{100, 100, 175, 200, 500}));
}
```

Operación asociativa no conmutativa.

```
// La concatenación asociativa comprueba el orden de operandos.
std::list<std::string> partes = {"a", "b", "c", "d"};
std::vector<std::string> prefijos;
parallel_inclusive_scan(partes.begin(), partes.end(),
    std::back_inserter(prefijos), std::plus<std::string>{}, 3);
assert((prefijos == std::vector<std::string>{
    "a", "ab", "abc", "abcd"}));
```

Rango vacío e iterador devuelto.

```
std::vector<int> vacio, salida;
auto fin = parallel_inclusive_scan(vacio.begin(), vacio.end(),
    std::back_inserter(salida), std::plus<int>{}, 4);
assert(salida.empty());
*fin++ = 7; // El iterador devuelto sigue siendo utilizable.
assert((salida == std::vector<int>{7}));
```

CS2023 - Programación III

Ejercicio de Programación Concurrente

Rubén Rivas Medina

Ejercicio 4: Primera coincidencia — Async · Intermedio

Una búsqueda puede localizar la primera lectura fuera de rango, el primer pedido vencido o la primera palabra que cumple un patrón. El predicado pertenece al llamador; el resultado debe ser el primer elemento según el orden del rango, aunque otra tarea termine antes.

Algoritmo y estrategia paralela

Implemente una variante paralela de `std::find_if`. Lance con `std::async(std::launch::async, ...)` una búsqueda por bloque contiguo. Cada tarea devuelve el primer iterador que satisface `pred` en su bloque, o el extremo del bloque. Combine los resultados en orden de bloque.

Interfaz

```
template<class ForwardIt, class Pred>
ForwardIt parallel_find_if(
    ForwardIt first, ForwardIt last,
    Pred pred, std::size_t tasks);
```

Restricciones

`ForwardIt` es al menos un iterador hacia adelante. Devuelva `last` si no hay coincidencia. `tasks == 0` lanza `std::invalid_argument`; el rango vacío devuelve `last`. Cree como máximo una tarea por elemento. Copie el predicado para cada tarea y suponga que esas copias pueden ejecutarse a la vez. Consuma todos los futuros antes de devolver o propagar una excepción. No comparta un iterador mutable entre tareas ni use `std::find_if` para implementar la búsqueda. El rango y sus elementos deben permanecer vivos y sin modificaciones hasta que termine la llamada.

Casos de prueba

Primera coincidencia con varios repartos.

```
std::vector<double> temperaturas = {18, 25, 21, 31};
for (std::size_t t : {1u, 3u, 8u}) {
    auto it = parallel_find_if(
        temperaturas.begin(), temperaturas.end(),
        [](double x) { return x > 24; }, t);
    assert(it == temperaturas.begin() + 1);
}
```

Otro tipo de iterador y ausencia de coincidencias.

```
std::list<std::string> estados = {"ok", "pendiente", "error"};
auto it = parallel_find_if(estados.begin(), estados.end(),
    [](const auto& s) { return s != "ok"; }, 2);
assert(it == std::next(estados.begin()));
assert(parallel_find_if(estados.begin(), estados.end(),
    [](const auto& s) { return s == "ausente"; }, 3)
    == estados.end());
```

Rango vacío.

```
std::vector<int> vacio;
assert(parallel_find_if(vacio.begin(), vacio.end(),
    [](int) { return true; }, 4) == vacio.end());
```